

Benchmarks and Statistics

Daniel Ari Friedman
ORCID: 0000-0001-6232-9096
Email: daniel@activeinference.institute
DOI: 10.5281/zenodo.16887791

October 07, 2026

Contents

1	Benchmarks and Statistics	1
1.1	Overview	1
1.2	Timing methodology	1
1.3	Percentile bootstrap	2
1.4	Pooled permutation tests	2
1.5	Effect size, multiplicity, and scaling	2
1.6	Jackknife, FDR, Welch, and circular statistics	3
1.7	A reproducible example	4
1.8	Cross-references	5

1 Benchmarks and Statistics

1.1 Overview

This section documents the measurement surface behind the preceding chapters. It is split across three modules with one job each: `src/quadmath/stats/benchmarks.py` measures (a small `perf_counter` timing harness over the core numerical surfaces), `src/quadmath/stats/statistics.py` analyzes (resampling inference, effect sizes, multiplicity correction, and scaling fits, on standalone `numpy`), and `src/quadmath/viz/vis_stats.py` renders (input-agnostic `matplotlib` primitives; the figures they produce are collected in `18_stats_gallery.md`). The measured workloads are the four surfaces the manuscript already treats: `quadray` conversions (`quadray.py`, conventions in `SPEC.md`), shell enumeration (`omni_numbering.py`, `13_lattice_tooling.md`), nearest-site search (`lattice_search.py`), and field fitting (`ivm_field.py`, `11_ivm_field_learning.md`). Everything here is deterministic: every random draw comes from a seeded `numpy.random.default_rng`, and every timing routine is fully specified by its arguments, so the numbers below reproduce bit-for-bit.

1.2 Timing methodology

`time_callable(fn, *, trials=5, warmup=1)` measures wall-clock duration with `time.perf_counter`, the highest-resolution monotonic clock available. It runs one warmup call first and discards it (first-call effects: module imports, key caches), then times `trials` calls and reports the summary statistics

$$\bar{t} = \frac{1}{T} \sum_{i=1}^T t_i, \quad t_m = \text{median}(t_1, \dots, t_T), \quad (1)$$

$$t_{95} = Q_{0.95}(t_1, \dots, t_T), \quad (2)$$

where T is the number of trials and Q_p the empirical p -quantile. The median is robust to a single outlier trial; the 95th percentile bounds the tail that a user would actually feel. Results are returned as `BenchRow`, a frozen dataclass with fields `name`, `n`, `trials`, `total_s`, `mean_s`, `median_s`, `p95_s`, and `ops_per_s`, plus `as_dict()` for plain data. Here n is the per-call workload size (conversions per call, shell depth, sites or queries per call), and throughput is

$$\text{ops_per_s} = \frac{n}{t}. \quad (3)$$

Four constructors wrap the core surfaces with these defaults: `bench_conversions(n=200, trials=5)` times round-trip quadray/embedding conversions, `bench_shell_enumeration(k_max=4, trials=5)` times shell enumeration through `omni_numbering.generate_shell`, `bench_lattice_search(n_sites=200, queries=50, trials=5)` times `lattice_search` nearest-site queries over a random ball of sites, and `bench_field_fit(n_sites=64, trials=3)` times a synthetic field fit through `IVMField.learn`. `run_all()` executes all four with the shared defaults collected in the module constant `BENCH_DEFAULTS` and returns the list of `BenchRow` records; `summary_table(rows)` renders them as a fixed-width text table. The harness measures only; it asserts nothing — timing distributions belong to analysis, not to test assertions.

1.3 Percentile bootstrap

`bootstrap_ci(x, stat=np.mean, *, iters=2000, seed=0, alpha=0.05)` gives a confidence interval for any statistic $\hat{\theta}$ by re-sampling the data with replacement. With $n = |x|$ and a seeded `numpy.random.default_rng(seed)`, iteration b draws resampling indices $i_{bj} \sim \text{Uniform}\{0, \dots, n-1\}$ in one vectorized call (`rng.integers(0, n, size=(iters, n))`) and evaluates

$$\hat{\theta}_b^* = \hat{\theta}(x_{i_{b1}}, \dots, x_{i_{bn}}), \quad \text{CI}_{1-\alpha} = \left[Q_{\alpha/2}(\hat{\theta}_{1..B}^*), Q_{1-\alpha/2}(\hat{\theta}_{1..B}^*) \right], \quad (4)$$

with $B = \text{iters}$. The percentile interval needs no distributional assumption about $\hat{\theta}$ — only that the empirical resampling distribution approximates its sampling distribution. All randomness is consumed from the seeded generator, so the interval is exactly reproducible.

1.4 Pooled permutation tests

`permutation_test(a, b, *, iters=2000, seed=0, alternative="two-sided")` tests whether two samples differ in location without assuming a distribution. Both samples are pooled; the observed statistic is $o = |\bar{a} - \bar{b}|$; each of the `iters` iterations draws `rng.permutation` of the pool, splits it at $|a|$ into a fake a and b , and recomputes the difference. The two-sided p-value uses the add-one convention

$$p = \frac{1 + \#\{r : |d_r| \geq o\}}{\text{iters} + 1}, \quad (5)$$

where d_r is the difference of permutation r . The $+1$ in numerator and denominator counts the observed arrangement itself and guarantees $p \geq 1/(\text{iters} + 1) > 0$: a permutation test can never report an impossible zero p-value, and the estimate is conservative. `alternative="greater"` and `"less"` use the same convention with signed comparisons of d_r against the signed observed difference.

1.5 Effect size, multiplicity, and scaling

`cohens_d(a, b)` reports the standardized mean difference

$$d = \frac{\bar{a} - \bar{b}}{s_p}, \quad s_p = \sqrt{\frac{(n_a - 1)s_a^2 + (n_b - 1)s_b^2}{n_a + n_b - 2}}, \quad (6)$$

with s_a, s_b the unbiased sample standard deviations. A p-value says whether a difference is distinguishable from noise; d says how large the difference is, which is the question that matters when comparing benchmark configurations.

`p_adjust_bonferroni(pvals)` controls the family-wise error rate when m hypotheses are tested at once:

$$p'_i = \min(1, m p_i). \quad (7)$$

Each p'_i can be read as a valid p-value at level α for the whole family — conservative, but assumption-free.

`scaling_fit(sizes, times)` fits ordinary least squares on the log-log data,

$$\log t = \beta_1 \log n + \beta_0, \quad (8)$$

and returns `(slope, intercept, r2)`. The slope is the empirical complexity exponent: $\beta_1 \approx 1$ indicates linear work in the input size n , $\beta_1 \approx 2$ quadratic, and the coefficient of determination r^2 measures how well the power law describes the measurements.

1.6 Jackknife, FDR, Welch, and circular statistics

`jackknife_ci(x, stat=np.mean, alpha=0.05)` produces both a confidence interval and a bias estimate for any scalar statistic without consuming a single random draw. Where the percentile bootstrap of (4) resamples with replacement, the jackknife deletes one observation at a time: each of the n leave-one-out samples is scored, and the spread of those scores determines the interval. With $\hat{\theta}$ the full-sample statistic and $\hat{\theta}_{(i)}$ the statistic on the sample with observation i deleted, the returned triple `(low, high, bias)` is

$$\begin{aligned} \text{bias} &= (n-1)(\bar{\theta}_{(\cdot)} - \hat{\theta}), & \hat{\theta}_{\text{corr}} &= \hat{\theta} - \text{bias}, \\ \text{se}_{\text{jack}} &= \sqrt{\frac{n-1}{n} \sum_{i=1}^n (\hat{\theta}_{(i)} - \bar{\theta}_{(\cdot)})^2}, & \text{CI}_{1-\alpha} &= \hat{\theta} \pm z_{1-\alpha/2} \text{se}_{\text{jack}}, \end{aligned} \quad (9)$$

where $\bar{\theta}_{(\cdot)}$ is the mean of the leave-one-out scores. The bias-corrected estimate $\hat{\theta}_{\text{corr}}$ removes the first-order bias that the jackknife detects in curved statistics; the interval itself stays centered on the uncorrected $\hat{\theta}$. The normal quantile z comes from bisecting `math.erfc(z / sqrt(2))`, which decreases monotonically from 2 to 0 as z runs from $-\infty$ to ∞ , so one hundred halvings of the bracket $[-40, 40]$ pin z to double precision. This deterministic `erfc`-bisection quantile needs no seed and no lookup table, so repeated calls with identical arguments return bit-identical results — the same reproducibility contract as the seeded routines above, but with no generator to seed at all.

`benjamini_hochberg(pvals)` controls a different error rate from the Bonferroni map of (7): instead of the family-wise error rate it controls the expected proportion of false discoveries among rejections, the false discovery rate. The step-up procedure sorts the m p-values ascending, forms the raw values $m p_{(i)}/i$ for ranks $i = 1, \dots, m$, and then enforces monotonicity with a running minimum taken from the largest rank backwards,

$$p'_{(i)} = \min\left(1, \min_{j \geq i} \frac{m p_{(j)}}{j}\right), \quad p'_{(m)} = \min(1, p_{(m)}), \quad (10)$$

so the adjusted values are monotone non-decreasing in the sorted p-values and never exceed the largest raw p-value. The backward cumulative minimum is the step that the naive $m p_{(i)}/i$ mapping misses: without it, a large early-rank raw value could exceed a smaller later-rank one, breaking monotonicity and invalidating the FDR guarantee. The stable argsort of the input is recorded so the adjusted values can be mapped back to the original input order, and the final cap at 1.0 keeps the output a valid p-value vector.

`welch_t_test(a, b, alternative="two-sided")` compares two sample means without assuming equal variances, the assumption that the pooled standard deviation s_p of `cohens_d` makes. Writing $w_a = s_a^2/n_a$ and $w_b = s_b^2/n_b$, the statistic and its Welch-Satterthwaite degrees of freedom are

$$t = \frac{\bar{a} - \bar{b}}{\sqrt{w_a + w_b}}, \quad \nu = \frac{(w_a + w_b)^2}{\frac{w_a^2}{n_a - 1} + \frac{w_b^2}{n_b - 1}}. \quad (11)$$

The p-value is evaluated from the exact Student- t tails rather than a normal approximation, and it needs no `scipy`: the two-sided tail uses the identity

$$P(|T| \geq |t|) = I_x\left(\frac{\nu}{2}, \frac{1}{2}\right), \quad x = \frac{\nu}{\nu + t^2}, \quad (12)$$

where $I_x(a, b)$ is the regularized incomplete beta function computed by a Lentz continued fraction (with the standard symmetry swap above $x = (a + 1)/(a + b + 2)$, an exp-log Γ front factor, and guard floors against vanishing denominators). "greater" and "less" reuse the same two-sided tail, halved on the appropriate side of $t = 0$. The degenerate branches matter for constant inputs: when both samples are constant the denominator of (11) vanishes, and equal means give $t = 0.0$ (p-value 1.0 two-sided) while different means give a signed infinite t whose tail probabilities are exactly 0 or 1, evaluated against the finite pooled fallback $\nu = n_a + n_b - 2$ so the tails stay well-defined.

`rotation_stats(angles)` treats angles in radians as points on a circle rather than points on a line, where 359° and 1° are neighbors:

$$\bar{\theta} = \text{atan2}\left(\frac{1}{n} \sum_j \sin \theta_j, \frac{1}{n} \sum_j \cos \theta_j\right), \quad R = \left| \frac{1}{n} \sum_j e^{i\theta_j} \right|. \quad (13)$$

The circular mean is the `atan2` of the averaged sine and cosine, wrapped into $(-\pi, \pi]$, and the mean resultant length $R \in [0, 1]$ measures concentration: $R = 1$ only when every angle agrees, $R = 0$ when the unit vectors cancel completely. The returned dictionary gives the circular variance $1 - R$ in $[0, 1]$, and `variance_2pi = 2(1 - R)` under the convention for angles on the full $[0, 2\pi)$ circle, which ranges over $[0, 2]$ and approaches the familiar linear variance for tightly clustered angles. Because sine and cosine are 2π -periodic, wrapping the angles into any full circle leaves every returned value unchanged; when the resultant vanishes the mean degenerates to whatever `atan2` returns for the cancelled components. Like everything else in `src/quadmth/stats/statistics.py`, all four routines are pure `numpy-plus-math` computations: no randomness, no optional scientific stack, and bit-identical repeats for identical arguments.

1.7 A reproducible example

Both workhorses above are fully determined by their arguments, so the following numbers are exact for any reader. The first call computes the 95% bootstrap confidence interval of the mean of the first seven Fibonacci numbers; the second asks whether the shifted ranges 1..10 and 11..20 differ in location:

```

1 import numpy as np
2
3 from quadmath.stats.statistics import bootstrap_ci, permutation_test
4
5 x = [2, 3, 5, 8, 13, 21, 34]
6 lo, hi = bootstrap_ci(x, np.mean, iters=2000, seed=0, alpha=0.05)
7 # (5.142857142857143, 20.571428571428573)
8
9 a = list(range(1, 11))
10 b = list(range(11, 21))
11 p = permutation_test(a, b, iters=2000, seed=0, alternative="two-sided")
12 # 0.0014992503748125937

```

With `bootstrap_ci(x, np.mean, iters=2000, seed=0, alpha=0.05)` the mean of x is $86/7 \approx 12.285714$ and the 95% percentile bootstrap interval is $[5.142857142857143, 20.571428571428573]$. Both endpoints are exact multiples of $1/7$: the mean of any 7-value resample with repetition is a multiple of $1/7$, so the percentile grid is discrete rather than dense. With `permutation_test(a, b, iters=2000, seed=0, alternative="two-sided")` on $a = 1, \dots, 10$ and $b = 11, \dots, 20$, the observed difference is $o = |\bar{a} - \bar{b}| = 10$ exactly, and exactly 2 of the 2000 seeded permutations reach it, giving

$$p = \frac{2 + 1}{2000 + 1} = \frac{3}{2001} \approx 0.00149925. \quad (14)$$

The count of 2 deserves a comment. Among the $\binom{20}{10} = 184,756$ possible splits of the pool, exactly two achieve $|d_r| = 10$ — the observed arrangement (1..10 against 11..20) and its complement — so the expected hit count over 2000 random draws is only $2000 \cdot 2/184756 \approx 0.02$. The add-one floor of (5) is $1/2001 \approx 0.0005$, which is what a typical seed reports here; the stream seeded with `seed=0` happens to draw both exact-maximum arrangements (at iterations 70 and 1146 of the 2000), so the count is 2 and the reported p-value is $3/2001$. The example illustrates the convention rather than a typical run: with any other seed the same call reproduces the floor value, and in either case the test correctly never reports the impossible $p = 0$.

The two interval families meet on common ground in a second example. Two synthetic lattice-error samples with a planted mean shift are drawn once from a seeded generator: routine A contributes $n = 64$ error residuals centered on 0.0 and routine B 64 residuals whose population mean sits 0.08 higher, both with scale 0.15 — the unit being whatever the error residual measures. For each sample mean, the 95% percentile bootstrap interval of (4) ($B = 2000$ resamples, re-seeded at 17) and the jackknife interval of (9) (which consumes no randomness at all) are computed, and the script `quadmath/scripts/stats_diagnostics_gallery.py` draws all four intervals in a single `plot_ciBars` call:

```
1 import numpy as np
2
3 from quadmath.stats.statistics import bootstrap_ci, jackknife_ci
4
5 rng = np.random.default_rng(17)
6 a = rng.normal(0.0, 0.15, size=64)
7 b = rng.normal(0.08, 0.15, size=64)
8 bootstrap_ci(a, np.mean, iters=2000, seed=17, alpha=0.05)
9 # (-0.07609532416913227, -0.0004865391750512773)
10 jackknife_ci(a, np.mean, alpha=0.05)[:2]
11 # (-0.07682589122849623, 0.0013292867175937334)
```

What the two families assume is where they differ. The percentile bootstrap needs no shape assumption at all — only that the empirical resampling distribution of (4) approximates the sampling distribution of $\hat{\theta}$ — while the jackknife interval is the normal approximation $\hat{\theta} \pm z_{1-\alpha/2} \text{se}_{\text{jack}}$ built from the n leave-one-out scores of (9). On the sample mean with $n = 64$ the two agree to within a whisker: routine A's bootstrap interval is $[-0.076095, -0.000487]$ against the jackknife's $[-0.076826, 0.001329]$, and routine B's is $[0.056788, 0.132418]$ against $[0.056011, 0.130639]$ (sample means -0.037748 and 0.093325 ; the jackknife bias estimates sit at floating-point zero, as (9) predicts for a linear statistic). The figure records the comparison: routine B's planted shift shows up as both of its intervals clearing the dashed zero line, while routine A hugs the line so tightly that the bootstrap upper endpoint stops 0.0005 short of it and the jackknife upper endpoint crosses it by 0.0013 — a hair-width disagreement that is the honest lesson of the panel. Coverage verdicts this close to the boundary are method-sensitive; a Welch test on the same samples reports $t \approx -4.75$ with $p \approx 5.4 \times 10^{-6}$ ((11)), in agreement with the shift both interval families detect.

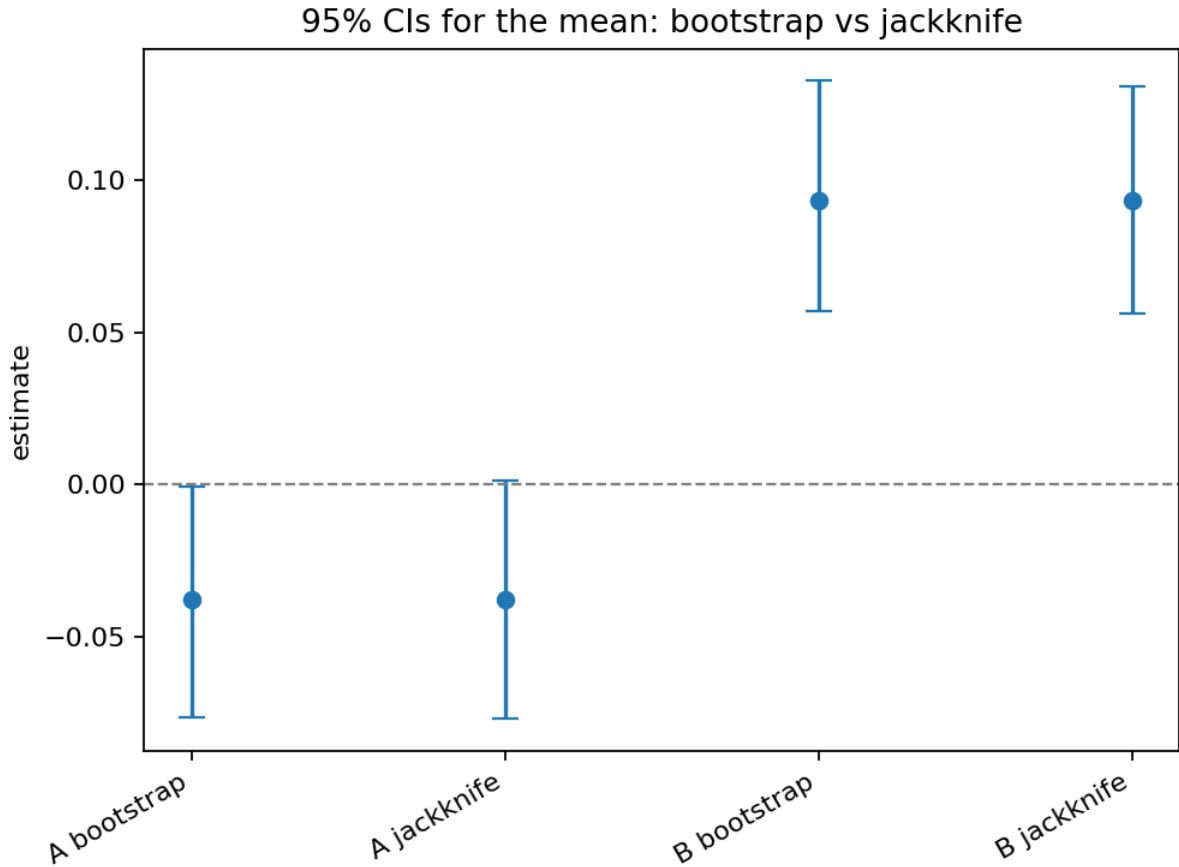


Figure 1: **Bootstrap versus jackknife 95% confidence intervals for two lattice-error samples.** Sample means (dots) of two seeded synthetic error samples — $n = 64$ per group from `numpy.random.default_rng(17)`, scale 0.15, planted mean shift 0.08 in the B population — each carrying a 95% percentile bootstrap interval (`bootstrap_ci`, 2000 resamples, seed 17, $\alpha = 0.05$) and a deterministic normal-approximation jackknife interval (`jackknife_ci`, $\alpha = 0.05$), drawn together in one `plot_ci_bars` call; the dashed line marks the unbiased population mean 0.0. Routine B's intervals exclude 0 while routine A's straddle it, and the two CI families agree within whisker width on both samples. Regenerate: `uv run python quadmath/scripts/stats_diagnostics_gallery.py`.

1.8 Cross-references

- Timing harness, BenchRow, and the four benchmark constructors: `src/quadmath/stats/benchmarks.py`.
- Bootstrap, permutation tests, `cohens_d`, `p_adjust_bonferroni`, `scaling_fit`: `src/quadmath/stats/statistics.py`.
- The figure primitives built on these results: `18_stats_gallery.md`.
- The interval-comparison script and figure behind the second example: `quadmath/scripts/stats_diagnostics_gallery.py` (`quadmath/output/figures/stats_ci_comparison.png`).
- Measured surfaces: `quadray.py` (conventions in `SPEC.md`), `omni_numbering.py` and `lattice_search.py` (`13_lattice_tooling.md`), `ivm_field.py` (`11_ivm_field_learning.md`).
- Sibling gallery of the lattice layer: `16_lattice_gallery.md`.